

苏州城市学院

课程论文

课程名称	物联网项目实践
学 院	计算科学与人工智能学院
班 级	23 物联网
学 号	2306443061
姓 名	吴遂杰
任课教师	石正璞
成 绩	

2026 年 09 月

目录

第一章	绪论	3
第二章	系统总体设计	5
第三章	MQTT 协议设计	8
第四章	ESP32 编程设计	13
第五章	基于 App Inventor 的安卓程序设计	29
第六章	Web 端的程序设计	32
第七章	系统联调与实验结果	35
第八章	总结	40

第一章 绪论

1.1 课程设计的背景与意义

物联网系统的典型形态是「感知层—网络层—应用层」的三层结构：感知层由各类传感器与执行器组成，负责物理量的采集与动作的执行；网络层把设备接入互联网并在设备与平台之间传递消息；应用层面向使用者，提供数据展示与远程控制。本次课程设计以《物联网综合实验平台》为载体，把嵌入式开发（ESP32）、安卓移动端开发（App Inventor）与 Web 端开发（HTML/CSS/JavaScript + MQTT.js）三条线合到一个题目上：以小组为单位，用 8 块 ESP32 模块搭出一套物联网多平台联动系统，并自行设计一套 MQTT 通信协议作为各端交互的唯一依据。

题目对本人的价值有三点。第一，必须真正理解 MQTT 的发布/订阅模型，以及主题、QoS、Retain、遗嘱消息等机制，否则协议设计无从下手。第二，必须把「设备端—服务器—多平台」整条链路打通，任何一环的细节（报文格式、时序、去抖、阻塞）出问题，现场演示就会失效。第三，协议必须先于代码定下来，多个终端按同一份文档并行开发，这与真实工程中「先定接口、再写实现」的做法完全一致。

1.2 小组选题与总体目标

本组共 4 人，使用实验箱上的 8 块 ESP32 模块。每块模块板接入一个（或一组）器件，8 块板各自是一个独立的 PlatformIO 工程，通过公网的 MQTT 服务器互联。整体目标是一套可以在课堂上现场演示的多设备联动系统，具备三项能力：

- ① 传感器数据实时上报，并在 Web 端与安卓 App 上同步显示；
- ② 平台可以对舵机、风扇、RGB 灯、蜂鸣器、语音播报五类执行器远程控制，设备执行后回发回执，平台据此确认动作是否真正完成；
- ③ 传感器与执行器之间在设备端直接联动，不经过平台，从而获得更快的响应速度，并在平台或网络异常时仍然可用。

为便于后文引用，先把 8 块板的编号、器件与设备标识列成对照表，如表 1-1 所示。表中「device」一列是报文里 body.device 的取值，「板号」既是下行报文中的 dst，也是上行报文中的 src。

表 1-1 各板编号与模块对照表

板号	模块名称	device	类型	涉及报文类型
1	舵机模块	servo	执行器	cmd、ack
2	继电器模块（风扇）	fan	执行器	cmd、ack
3	红外对射传感器模块	ir_beam	传感器	evt、dat
4	温湿度传感器模块（SHT30）	sht30	传感器	dat
5	ADC/RGB/蜂鸣器综合模块	rgb、buzzer	综合	cmd、ack、evt、dat
6	火焰传感器模块	flame	传感器	evt、dat
7	智能语音播报模块	tts	执行器	cmd、ack

板号	模块名称	device	类型	涉及报文类型
8	光照度传感器模块 (BH1750)	light_sensor	传感器	dat

1.3 本人承担的工作

本人在组内负责三部分工作：一是《MQTT 通信协议规范_第 1 组》的编写与后续订，包括主题设计、报文格式、各模块数据点定义以及对标作业要求的字段对应关系；二是 1 号板（舵机，device 为 servo）与 7 号板（智能语音播报，device 为 tts）两块板固件的开发，两块板分别代表「严格参数校验 + 非阻塞动作」与「带优先级的语音播报队列」两类典型问题；三是对其余六块板固件的整合与统一管理，包括把命名、分层、配置方式归一，把协议层代码抽成八份逐字节一致的副本，并把分散的模块说明合并为一份《功能与设计说明》。因此第四章会以 1 号板、7 号板与协议层为主；第五章、第六章介绍组内其他成员完成的安卓端与 Web 端，第七章给出整组联调的实测结果。

第二章 系统总体设计

2.1 系统架构图

本系统按「感知/执行层—网络层—应用层」三层组织，整体架构如图 2-1 所示。下面是 8 块 ESP32 模块，每块板上跑一个独立固件；中间是一台公网 MQTT 服务器（yunyismart.tech），它不保存业务逻辑，只按主题把报文转发给订阅者；最上面是 Web 端与安卓 App 两个平台端。设备与平台之间没有任何直连，双方都只与服务器打交道，因此只要主题与报文格式不变，任何一端改动都不影响另一端。

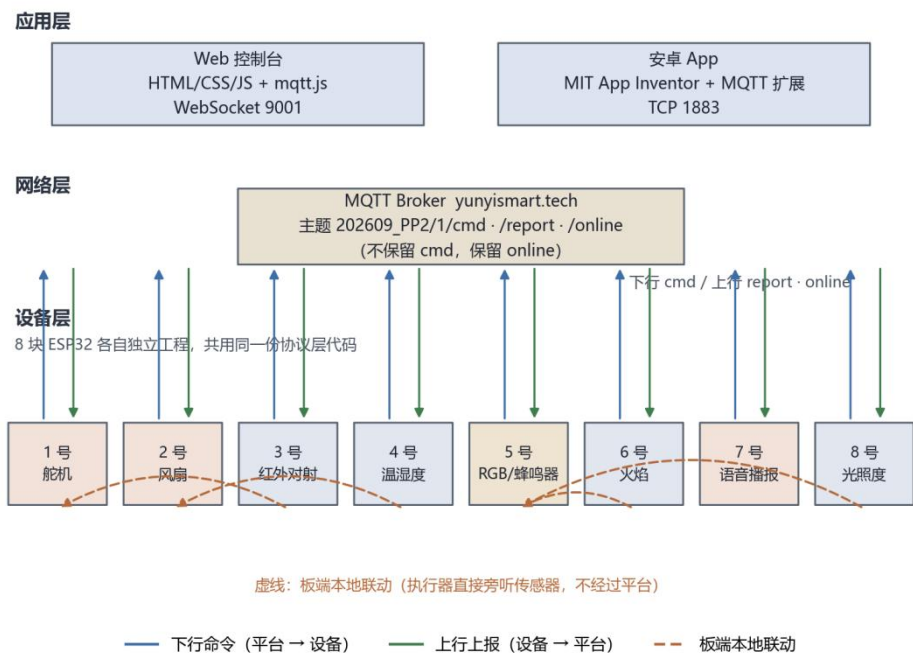


图 2-1 系统总体架构与数据流向

三层之间一共只有三条 MQTT 主题：下行的 202609_PP2/1/cmd、上行的 202609_PP2/1/report 与 202609_PP2/1/online。前缀 202609_PP2/1 由课程空间前缀 202609_PP2 与本组组号 1 组成，这样不同小组即使连到同一台服务器也不会互相收到报文。

除三层结构之外，系统中还有一条不经过平台的旁路：1 号、2 号、5 号、7 号板额外订阅了 report 主题，直接旁听传感器板的上报报文，自行判断并驱动自己的执行器。图 2-1 中用虚线表示这四条板端联动，即 3 号对射→1 号舵机、4 号温湿度→2 号风扇、8 号光照→5 号 RGB、6 号火焰→5 号 RGB 与蜂鸣器。

2.2 功能模块与小组分工

本组采用「按板块划分、协议与文档共同编写」的分工方式，具体原则有三条。第一，8 块板按硬件接口关系分成四组，每名成员负责一组（2 块板）的固件开发、单独调试与对应的模块说明文档，做到「谁写的板谁最熟」。第二，通信协议与整组代码的命名、分层、配置规范由一人统一维护，其他人改动协议层代码必须同步全部八份副本，避免八块板之间出现「看起来一样、实际上是两个版本」的情况。第三，Web 端与安卓端各由一名成员负责，两端只与 MQTT 服务器通信，因此可以与其他人的固件开发并行推进。

这种分法的依据是模块之间的耦合程度。传感器板与执行器板的代码结构几乎相同（初始化、周期采集或命令处理、上报），放在一起由同一人完成，可以一次把公共的工程结构定下来，其余板直接套用；而协议层、Web 端、安卓端与任何一块具体的板都无关，单独安排反而不容易互相等待。表 2-1 给出各部分的职责、交付物与彼此的依赖关系。

表 2-1 各部分职责与交付物

工作包	主要内容	交付物	依赖
协议与代码规范	主题、报文格式、数据点定义；八块板命名与分层统一	《MQTT 通信协议规范_第 1 组》V2.0、《功能与设计说明》	无
执行器板固件（组一）	1 号舵机、2 号风扇	两套 PlatformIO 工程	协议
传感器板固件（组二）	3 号对射、4 号温湿度	两套工程	协议
综合板与语音板（组三）	5 号 RGB/蜂鸣器、7 号语音播报	两套工程（含板端联动与三层播报）	协议
传感器板固件（组四）	6 号火焰、8 号光照度	两套工程	协议
安卓端	App Inventor 界面与逻辑块，发布命令、显示数据与事件	group01_final.aia、group01_final.apk	协议
Web 端	HTML/CSS/JS + mqtt.js 控制台，含联动调试面板	index.html、style.css、script.js	协议

需要说明的是，板端联动把「传感器—执行器」的关系放在了固件里，因此参与联动的两块板必须由同一个人或同一套约定来维护。本组的做法是把联动阈值与时序参数全部集中在各自的 my_config.h 中，并在《功能与设计说明》里列成规则表，任何人改动都能一眼看出影响了哪条链路。

2.3 多平台联动设计

多平台联动的本质是「同一条报文被多个订阅者按同一套规则解读」。以一次典型的控制为例：在 Web 端把舵机角度滑块拖到 90° 并点击「设定」，Web 端会往 cmd 主题发布一条报文；服务器把它转发给订阅了 cmd 的全部 ESP32；只有 dst 等于自身板号的 1 号板处理这条命令，其余板直接丢弃；1 号板完成平滑转动后往 report 主题回发 ack；Web 端与安卓端都订阅了 report，两边同时读到这条回执并把界面更新为「已转到 90°」。整条链路如图 2-2 所示。

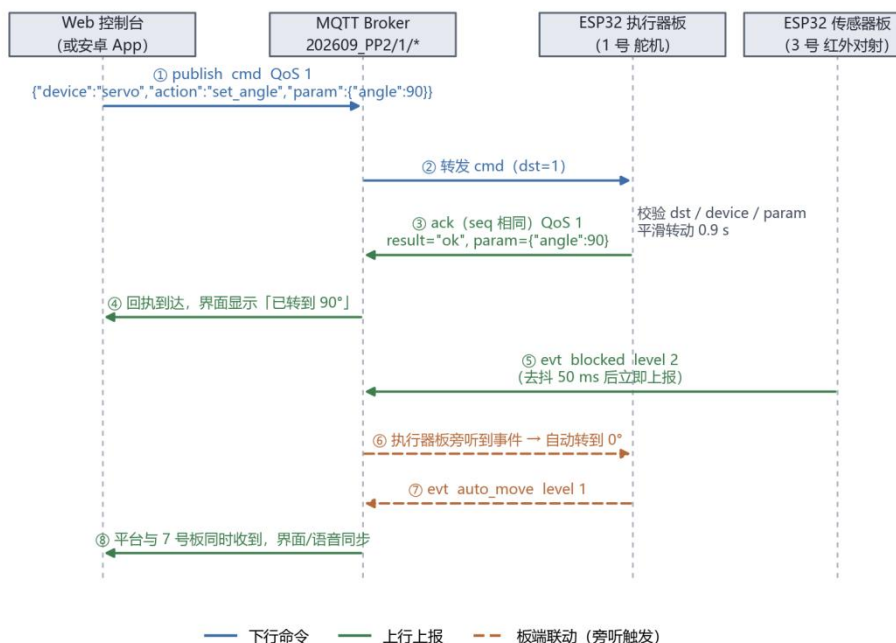


图 2-2 一次完整控制与上报链路的报文时序

上行方向同理。4 号板每 5 秒把温度与湿度合并成一条 dat 报文发布到 report 主题，Web 端与安卓端都会收到，两边的读数因此天然保持一致，不需要任何额外的同步机制。设备上下线走 online 主题，该主题带 Retain 标志，后订阅的平台能立即读到各板的最新状态，不必等到下一次上线报文。

把联动放在设备端还是平台端，本组做了两次实现并做了对比。按协议第七章给出的做法，传感器上报后由平台判断并下发命令，优点是规则集中、改起来方便；缺点是多了一次「上报→判断→下发」的往返，实测延时在 1 秒左右，而且平台没写对或网络抖动时联动就失效。本组因此在固件端又实现了一套等效的联动：执行器板旁听传感器报文后自行动作，延时在 1 秒以内，断网也不影响。两套机制并不冲突——平台随时可以下发命令覆盖联动的结果，而当板端发生联动时，执行器会额外发出一条扩展事件（如 fan 的 auto_on、rgb 的 scene_fire），平台与 7 号板据此就能知道「这次动作是联动触发的，不是人点的」。

第三章 MQTT 协议设计

本章内容与本组提交的《MQTT 通信协议规范_第 1 组》V2.0 一致，该文档由本人编写并维护。协议分为三部分：主题设计、报文格式与字段定义、各模块数据点定义。

3.1 主题设计

全系统只用三条主题，收发关系与服务质量如表 3-1 所示。

表 3-1 主题定义（前缀统一为 202609_PP2/1）

主题后缀	方向与收发方	QoS / Retain
cmd	下行：各平台 → 各 ESP32 板	QoS 1，不保留
report	上行：各 ESP32 板 → 各平台	QoS 0 (dat) 或 1 (evt、ack)，不保留
online	上行：各 ESP32 板 → 各平台、各板	QoS 1，保留

主题设计主要有四处考虑。第一，下行控制与上行上报分成两个主题，ESP32 只订阅 cmd，不订阅 report，因此不会收到任何板子上报的数据，既避免了自身报文被自己重新解析（自回环），也减少了无效报文的处理开销；各平台则可以直接按主题区分报文方向。第二，cmd 主题不使用 Retain 标志——若使用保留消息，设备断线重连后会立刻收到服务器保存的最后一条命令并重新执行，可能造成舵机、继电器等执行器误动作。第三，online 主题使用 Retain 标志，各平台订阅后能立即读到每块板的最新在线状态。第四，调试时可以用通配符 202609_PP2/1/# 一次订阅本组全部主题，该通配符按层级匹配，不会收到其他组的数据。

3.2 报文格式与字段定义

所有 MQTT 消息的负载必须是 JSON，采用「信封 + 载荷」两层结构：信封字段对所有报文完全相同，载荷的内部结构则由报文类型决定。一条完整的下行命令与它对应的回执如下（为便于排版按字段分行，实际发送时换行可以任意增删）。

代码 3-1 下行命令与执行回执（协议 § 3.1 示例，可在 MQTTX 中直接使用）

```
{ "ver": "1.0", "type": "cmd", "seq": 12, "src": "web", "dst": 1,
  "body": { "device": "servo", "action": "set_angle", "param": { "angle": 90 } } }

{ "ver": "1.0", "type": "ack", "seq": 12, "src": "1", "dst": 0,
  "body": { "device": "servo", "action": "set_angle", "result": "ok",
    "param": { "angle": 90 } } }
```

报文固定包含 ver、type、seq、src、dst、body 共 6 个字段，不得增减，字段名一律小写。两条报文的 seq 相同，平台据此把回执与前面的命令对应起来。各字段的含义与取值如表 3-2 所示。

表 3-2 信封字段定义

字段	类型	说明
ver	字符串	协议版本，本版本固定为“1.0”；版本不一致时忽略该报文并在串口提示
type	字符串	报文类型，取值为 cmd、ack、dat、evt、sys 之一
seq	整数	报文序号，由发送方从 1 开始自增，到 65535 后回到 1；ack 沿用所回应命令的 seq
src	字符串	发送方标识：平台端为“web”或“app”，设备端为板号字符串（如“3”）
dst	整数	目标板号 1~8；0 表示广播。上行报文的 dst 固定为 0
body	对象	载荷，内部结构由 type 决定

信封的设计目的是让「判断这条报文是不是给我的、是什么类型」这件事只依赖固定的 6 个字段。任一终端收到报文后先解析信封，再决定是否继续解析 body，各端的解析入口因此可以完全一致，不必为每种设备单独写识别分支。特别地，设备标识与板号相互独立：dst 决定由哪块板处理，body.device 决定由该板上的哪个模块处理，5 号板同时挂载 rgb 与 buzzer 时不需要增加额外字段。

五类报文的分工如表 3-3 所示，各类报文的 body 结构如表 3-4 所示。

表 3-3 报文类型定义

type	名称	方向	说明
cmd	命令	下行	平台要求设备执行动作，必须携带 body.action；设备无论执行成功与否都必须回 ack
ack	回执	上行	设备对命令的执行结果，result 取“ok”或“fail”；seq 与所回应的命令一致
dat	数据	上行	传感器周期采集的数据，一次可携带多个数据点
evt	事件	上行	状态变化与告警事件，如光束遮挡、检测到火焰；状态变化时立即上报
sys	系统	上行	设备自身的运行状态，本版本用于上线下线通知

表 3-4 各报文类型的 body 字段

type	body 内字段	说明
cmd	device、action、param	param 为参数对象，无参数时可省略
ack	device、action、result、param	param 回发执行后的实际状态
dat	device、samples	samples 是数组，元素形如 {"key":..., "value":...}

type	body 内字段	说明
evt	device、event、level	level 为事件等级 0~3，省略时按 1 处理
sys	device、event、uptime	device 固定为 "sys"，event 取 online 或 offline

取值与类型约定如表 3-5 所示。参数一律使用 JSON 原生类型：数值不加引号，开关使用布尔值，只有文本才是字符串。这样各平台不需要再做字符串到数值的转换，也不会出现不同平台对同一字符串解析方式不一致的问题。

表 3-5 取值与类型约定

类别	取值	说明
布尔	true / false	开关类参数，如 {"on": true}，不加引号
角度	0~180 的整数	舵机角度，单位为度
温度	-40.0~125.0	单位为 ℃，保留 1 位小数
湿度	0.0~100.0	相对湿度，单位为 %RH，保留 1 位小数
光照度	0.0~65535.0	单位为 lx，保留 1 位小数
模拟量	0~4095 的整数	12 位 ADC 与火焰强度的原始值
时长	50~5000 的整数	蜂鸣器鸣叫时长，单位为毫秒
事件等级	0~3 的整数	0 恢复，1 提示，2 警告，3 紧急

作业要求协议「至少包含设备标识、设备编号、控制动作、反馈数据」，本协议与这四项要求的对应关系如表 3-6 所示。

表 3-6 作业要求与本协议的字段对应

作业要求	本协议中的对应字段
设备标识	body.device，如 "servo"、"rgb"、"flame"
设备编号	下行报文的 dst，上行报文的 src，取值 1~8，0 为广播
控制动作	type 为 "cmd" 的报文中的 body.action 与 body.param
反馈数据	type 为 "ack" 的 result、param；"dat" 的 samples； "evt" 的 event、level

3.3 各模块数据点定义

各模块的数据点按板号逐个定义，方向一栏中「下行」表示由平台发给 ESP32，「上行」表示由 ESP32 发给平台。为节省篇幅，表 3-7 只列出核心数据点，完整的取值说明、上报周期与去抖参数见协议文档第五章与表 19。

表 3-7 各模块核心数据点

板号 / device	方向与 type	body 内容	说明
1 servo	下行 cmd	action:"set_angle"、 param:{"angle":0~180}	舵机转到指定角度

板号 / device	方向与 type	body 内容	说明
1 servo	上行 ack	action:"set_angle"、 result:"ok" / "fail"、 param:{angle}	回发实际角度
2 fan	下行 cmd	action:"set_power"、 param:{"on":true/false}	继电器吸合 / 断开
3 ir_beam	上行 evt	event:"blocked" level 2 / "clear" level 0	遮挡立即上报, 30 秒心跳
3 ir_beam	上行 dat	samples:[{"key":"count", "value":12}]	累计遮挡次数 (扩展数据点)
4 sht30	上行 dat	samples:[{temperature, 26 .5}, {humidity, 60.2}]	温湿度合并一条, 每 5 秒
5 rgb	下行 cmd	action:"set_color"、 param:{"r","g","b"}	三通道各 0~255, 全 0 即熄灭
5 buzzer	下行 cmd	action:"set_power"、 param:{"on":true}	持续鸣叫 / 停止
6 flame	上行 evt	event:"detected" level 3 / "clear" level 0	检测到火焰立即上报, 持续期间 每 5 秒重发
6 flame	上行 dat	samples:[{"key":"intensi ty","value":3120}]	火焰红外强度, 每 1 秒
7 tts	下行 cmd	action:"play_text"、 param:{"text":播报文本}	文本不超过 100 字节
8 light_sen sor	上行 dat	samples:[{"key":"illumin ance","value":125.5}]	光照度, 每 2 秒

几处命名与取值上的约定值得单独说明。红外对射命名为 ir_beam、火焰传感器命名为 flame：两者都属于红外检测类元件，但检测对象与告警含义完全不同，若统一命名为 ir_sensor，平台端无法区分。继电器驱动的负载按实际功能命名，本组继电器接风扇，故 device 为 fan，平台界面可以直接对应「风扇」控件，将来更换负载只需调整代码，报文格式不变。RGB 不单独定义开关命令，param 中三通道全为 0 即表示熄灭，于是 rgb 只有一种参数形式，平台端无需区分「开灯」与「设颜色」两种状态。

3.4 QoS 与 Retain 的取舍

QoS 与 Retain 的取值如表 3-8 所示，依据是「这条报文丢了会不会造成不可接受的后果」。控制命令必须送达，因此用 QoS 1；周期数据 (dat) 允许少量丢包，用 QoS 0 以降低网络与服务器开销；事件与回执 (evt、ack) 反映了状态跳变，一块板只发生一次，丢了就再也补不回来，因此也用 QoS 1。

表 3-8 QoS 与 Retain 取值

主题与报文类型	QoS	Retain	理由
cmd 全部命令	1	否	控制命令必须送达；不保留，避免重连后重复执行历史命令
report 周期数据 dat	0	否	允许少量丢包，降低网络与服务器开销
report 事件与回执 evt、ack	1	否	状态跳变只发生一次，不允许丢失
online 在线状态 sys	1	是	后订阅的平台能立即读到各板最新状态

这里有一个实现层面的约束：课程例程使用的 PubSubClient 库，其 publish() 只能发 QoS 0，达不到协议对 QoS 1 的要求（MQTTClient 库的 PublishEx 功能才能指定 QoS）。为此八块板统一改用 256dpi/arduino-mqtt。该库的收发缓冲区默认只有 128 字节，装不下带 param 的 ack 与 dat 报文，所以必须在构造时显式放大为 MQTTClient client(1024, 1024)，否则长报文会被静默截断——现象是命令发出去了、板子却毫无反应，且没有任何错误提示。

第四章 ESP32 编程设计

本章介绍设备端固件的实现。由于本人在组内承担协议层、1 号板舵机与 7 号板语音播报的开发，并负责其余六块板的整合，本章按「协议公共层 → 舵机 → 语音播报 → 传感器模块的统一整合」的顺序展开。

4.1 工程结构与协议层的抽离

8 块板各自是一个独立的 PlatformIO 工程，可以单独编译、单独烧录，目录结构完全一致：

代码 4-1 单个模块工程的目录结构（以 7 号板为例，源码/ESP32 嵌入式开发/智能语音_7 号）

```
智能语音_7 号/
platformio.ini      工程配置（开发板、依赖库、串口速率）
README.md          本模块的功能说明与 MQTTX 调试指南
src/
  my_config.h       配置：板号、设备名、主题、引脚、阈值、宏开关
  mqtt_proto.h      协议公共层：接口声明
  mqtt_proto.cpp     协议公共层：实现（唯一依赖 MQTT 库的文件）
  tts.h / tts.cpp    本模块的业务逻辑
  linkage.h / .cpp   本地联动的规则（只有 1、2、5、7 号板有）
  main.cpp          程序入口、命令分发
```

八个工程共用同一份协议层代码：mqtt_proto.h 与 mqtt_proto.cpp 在八个工程里是逐字节一致的副本，八块板之间的差异全部由各自的 my_config.h 用宏控制。这样做的直接好处是，协议层的任何修正（例如回执时机、报文格式）只需写一次再同步七份，不必在八个文件里各改一遍；而代价是必须保证同步，因此每个副本的文件头都写明了这条约束。本人负责的整合工作主要就是把改造前的八个独立工程统一到这套结构上：统一命名、统一分层、把协议层合并为同一份，并把各板的阈值、引脚、时序参数集中到 my_config.h。

4.2 协议公共层：解析、分流与回执

协议层只做四件事：连接 WiFi 与 MQTT、把收到的报文按协议解析并分流、维护发送序号、按报文类型组包并发布。它是全工程唯一依赖 MQTT 库的地方，将来若要换库，只改这一个文件。报文解析的入口是 on_mqtt_message()，其核心逻辑如下。

代码 4-2 报文的信封校验与分流（源码/ESP32 嵌入式开发/智能语音_7 号 /src/mqtt_proto.cpp, on_mqtt_message() 节选）

```
const char *ver = doc["ver"] | "";
const char *type = doc["type"] | "";
const char *src = doc["src"] | "";

// 协议 §3.2: 先判断版本，不一致就忽略并在串口提示
if (strcmp(ver, PROTO_VER) != 0)
{
```

```

        Serial.printf("[proto] 协议版本不符（收到 \"%s\"），忽略\n", ver);
        return;
    }

    // 【防自回环】本板自己发的报文一律丢掉。
    if (atoi(src) == BOARD_ID)
        return;

    JsonObjectConst body = doc["body"].as<JsonObjectConst>();

    // 非 cmd 报文：本层不处理，交给可选钩子
    if (strcmp(type, "cmd") != 0)
    {
        proto_on_other(type, src, body);
        return;
    }

    // dst 为目标板号，0 表示广播
    int dst = doc["dst"] | -1;
    if (dst != 0 && dst != BOARD_ID)
        return;

```

这段代码体现了协议约定的解析顺序：先读信封判断「是不是给我的、是什么类型」，再按 type 分支，最后按 body.device 分发到具体模块。其中 src 过滤这一句是板端联动的安全保证——1、2、5、7 号板为了联动额外订阅了 report，会收到其他板的报文，但自己发出的报文在解析入口就被丢弃，所以不可能形成回环。

分流之后的动作处理交给各板的 main.cpp，协议层与业务代码之间用两个弱符号钩子解耦，接口声明与默认实现如下。

代码 4-3 协议层与业务代码之间的接口与弱符号钩子（源码/ESP32 嵌入式开发/智能语音_7号/src/mqtt_proto.h 与 mqtt_proto.cpp）

```

// mqtt_proto.h: 由各板的 main.cpp 实现，把命令分发给本板的设备模块
ProtoResult handle_device_cmd(const char *device, const char *action,
                              JsonObjectConst param, JsonObject ackParam);

// 本板执行器是否还在动作（默认返回 false）
bool device_busy();

// 收到非 cmd 报文（evt / dat / sys）时被调用
void proto_on_other(const char *type, const char *src, JsonObjectConst body);

// mqtt_proto.cpp: 弱符号的默认实现
__attribute__((weak)) bool device_busy()
{

```

```

    return false;
}

__attribute__((weak)) void proto_on_other(const char *type, const char *src,
                                          JsonObjectConst body)
{
    (void)type;
    (void)src;
    (void)body;
}

```

弱符号的意义在于：不实现这两个函数的板沿用默认行为，与改造前完全一致；需要特殊行为的板在自己的模块文件里定义同名强函数即可覆盖。1 号板在 `servo.cpp` 里覆盖了 `device_busy()`，用来把 `ack` 延后到舵机停稳之后；7 号板在 `linkage.cpp` 里覆盖了 `proto_on_other()`，用来做本地联动播报。其余六块板一行都不用改。

回执的发送时机由 `flush_pending()` 控制，这是协议层里最需要小心的部分：它既要保证「设备无论执行成功与否都必须回 `ack`」，又要允许执行器把回执推迟到动作真正完成之后。

代码 4-4 回执的即时回、失败回与挂起后补发（源码/ESP32 嵌入式开发/智能语音_7 号 /src/mqtt_proto.cpp, `flush_pending()` 节选）

```

#ifdef ACK_AFTER_DONE
    // 有一条命令已经受理、执行器还在动作：等它做完再把 ack 发出去。
    if (wait_ack)
    {
        if (device_busy())
            return; // 还没到位，下一轮再来看

        send_ack(wait_device, wait_action, "ok", wait_seq,
                  wait_param.as<JsonObject>());
        wait_ack = false;
        return;
    }
#endif

    // .....（略去 pending 判断与 handle_device_cmd() 的调用）.....

    if (r == PROTO_FAIL)
    {
        // 参数非法：立即回 fail，不涉及执行器动作，没有挂起的必要
        send_ack(pend_device, pend_action, "fail", pend_seq, ackParam);
        return;
    }
}

```

```

#if ACK_AFTER_DONE
// 命令已受理，但执行器还在动作：把回执挂起，到位置了再发
if (device_busy())
{
    // ..... (略去 wait_device / wait_action 的字符串拷贝) .....
    wait_seq = pend_seq;
    wait_param.clear();
    wait_param.to<JsonObject>().set(ackParam);
    wait_ack = true;
    // ..... (略去串口日志) .....
    return;
}
#endif

send_ack(pend_device, pend_action, "ok", pend_seq, ackParam);

```

挂起期间不再受理新命令，避免两条命令的回执交错；补发时使用的仍是原命令的 seq，因此平台仍然能把回执与命令正确配对。参数校验失败的命令不挂起，立刻回 fail——它本来就没有让执行器动起来，没有等待的理由。协议层的完整流程如图 4-1 所示。

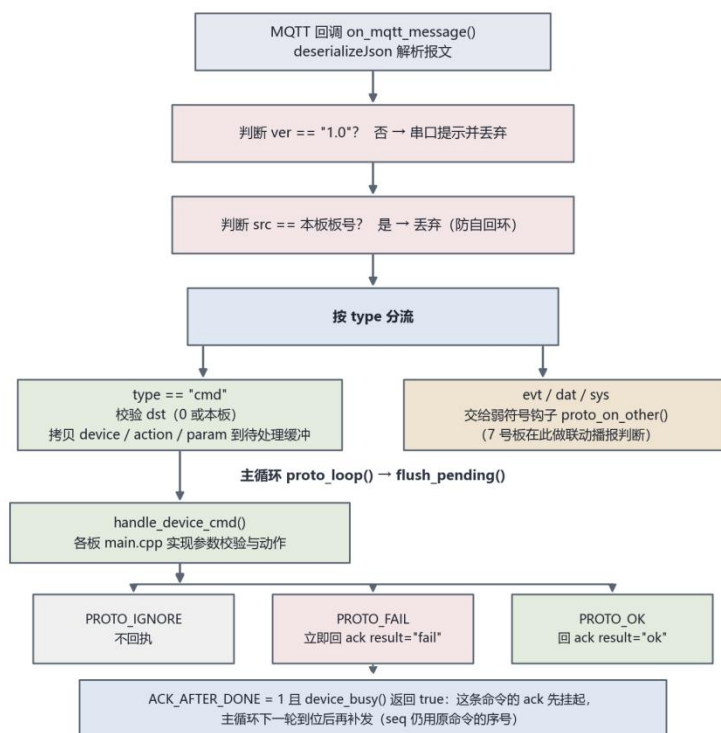


图 4-1 协议公共层的解析、分流与回执流程

4.3 1 号板舵机模块：严格校验与非阻塞动作

舵机模块有两个与执行器身份直接相关的设计：参数校验从严，以及转动过程不能阻塞主循环。参数校验的实现在 main.cpp 的 read_angle() 里。

代码 4-5 舵机角度的严格校验（源码/ESP32 嵌入式开发/舵机_1 号/src/main.cpp, read_angle()）

```
static bool read_angle(JsonObjectConst param, int *out)
{
    if (!param.containsKey("angle"))
        return false;

    JsonVariantConst v = param["angle"];

    // 只接受数值：布尔 true 会被当成 1、字符串 "90" 会被当成 90，
    // 这两种都必须挡掉（协议表 5：角度是 JSON 原生数值，不加引号）
    if (v.is<bool>() || (!v.is<int>() && !v.is<float>()))
        return false;

    float f = v.as<float>();
    if (f != (float)(int)f)
        return false; // 非整数，例如 90.5

    if (f < (float)SERVO_ANGLE_MIN || f > (float)SERVO_ANGLE_MAX)
        return false; // 越界

    *out = (int)f;
    return true;
}
```

校验不通过时返回 PROTO_FAIL，协议层回一条 result 为 fail 的 ack，舵机保持原位不动，绝不「缺了就按 0 处理」。对执行器来说，把 135° 误当成 0° 直接甩过去会打到限位、烧坏齿轮，代价远大于不动作，因此参数校验与动作执行必须在同一步完成，校验不过就不进入执行分支。同样的原则也用在 5 号板的 RGB 上：三个通道必须都存在、都是数值、都在 0~255 之间，否则回 fail 且灯保持原状，避免一个残缺报文把灯误熄灭。

第二个设计是非阻塞转动。最容易想到的写法是用 for 循环加 delay 一次转到位，但舵机转 180° 需要约 1.8 秒，主循环里跑着 MQTT 心跳与收发，被卡住 1.8 秒意味着期间收不到新命令、回执全部延迟，卡久了还会被服务器判超时踢下线。因此转动被拆成「每 10 毫秒走 1 度」的小状态机，servo_loop() 每次只走一小步就返回。

代码 4-6 舵机的非阻塞转动与动作状态查询（源码/ESP32 嵌入式开发/舵机_1 号/src/servo.cpp, servo_loop() 与 device_busy()）

```
void servo_loop()
{
    if (cur_deg == tgt_deg)
        return; // 已到位，什么都不做

    uint32_t now = millis();
```

```

    if (now - last_step_ms < SERVO_STEP_MS)
        return; // 还没到下一小步的时刻
    last_step_ms = now;

    led_moving(true); // 转动期间点亮

    // 走一小步，最后一步收到目标角度上，不会冲过头
    if (tgt_deg > cur_deg)
        cur_deg = min(cur_deg + SERVO_STEP_DEG, tgt_deg);
    else
        cur_deg = max(cur_deg - SERVO_STEP_DEG, tgt_deg);

    srv.write(cur_deg);

    if (cur_deg == tgt_deg)
    {
        led_moving(false);
        Serial.printf("[servo] 到位 %d°\n", cur_deg);
    }
}

// 覆盖协议层 mqtt_proto.cpp 里的弱函数：返回 true 时协议层把 ack 挂起
bool device_busy()
{
    return servo_busy();
}

```

这段代码与代码 4-4 是配套的：servo_busy() 返回 cur_deg != tgt_deg，协议层收到命令后把它作为 device_busy() 的结果，于是「转动到位才回执」这件事由协议层统一完成，业务代码只需如实回答自己是否还在动。图 4-2 给出了这条链路的完整时序。

主循环每轮只走一小步（每 10 ms 走 1°），全程不阻塞 MQTT 收发

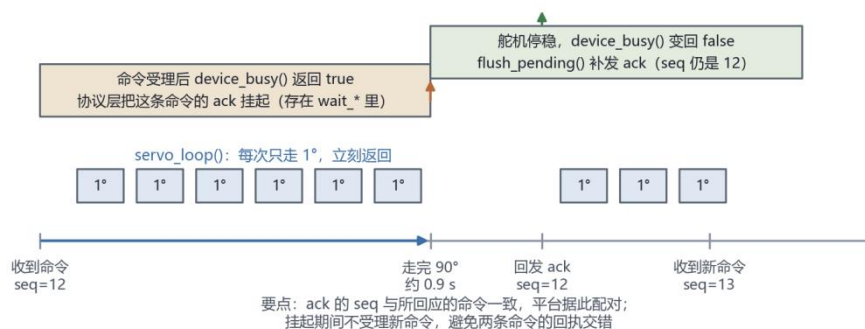


图 4-2 舵机非阻塞转动与延迟回执的时序

1 号板还有一条板端联动：旁听 3 号板的光束遮挡事件，自行把舵机转到遮挡位（0°），光束恢复后回到中位（90°）。实现要点是只认状态跳变——3 号板在状态不变时每 30 秒发一次心跳事件，那些心跳与触发无关，不能被当成新的遮挡。

代码 4-7 舵机的对射联动与联动事件上报（源码/ESP32 嵌入式开发/舵机_1 号 /src/linkage.cpp, linkage_loop()）

```
void linkage_loop()
{
    if (!need_update)
        return;

    // 舵机还在转就等它停稳，不要中途改目标。
    if (servo_busy())
        return;

    need_update = false;

    int want = ir_blocked ? LINK_SERVO_BLOCKED_DEG : LINK_SERVO_CLEAR_DEG;

    // 已经在目标位置：既不再转一次，也不再上报一条 evt，
    // 免得平台看到一串没有实际动作的 auto_move。
    if (servo_angle() == want)
        return;

    servo_set_angle(want);

    // 把联动动作广播出去：平台能看是“谁让它转的”，
    // 7 号板收到后也会播报一句。
    proto_send_evt(DEV_SERVO, ir_blocked ? "auto_move" : "auto_home",
```

```

        ir_blocked ? 1 : 0);

    Serial.printf("[link] 联动转动到 %d°\n", want);
}

```

联动动作会以扩展事件（servo 的 auto_move / auto_home）发布到 report 主题，平台据此把「联动触发」与「人点按钮」区分开，7 号板收到后也会播报一句。这类报文属于协议 8.6 节允许的扩展数据点，不影响任何标准报文。

4.4 7 号板智能语音播报模块：转码、组帧与优先级队列

7 号板是本组功能最复杂的执行器，它有两条播报来源：平台下发的 play_text（标准路径），以及本地联动——自己订阅 report 与 online，把其他板的运行情况翻译成人话播报出来。两条路径最终都调用同一个 tts_say() 入队，由 tts_loop() 统一下发。

下发前有两处必须处理：编码转换与超长截断。MQTT 报文按协议用 UTF-8 传输，而语音模块只认 GB2312，少这一步模块收到的是乱码，现象是「有声音但念的是杂音」，比完全不响更难排查；模块的缓冲区有限，协议 §5.7 建议播报文本不超过 100 字节，超长必须截断，而且截断点要切在字符边界上——GB2312 里汉字是双字节，按字节硬截断有可能把一个汉字劈成两半，模块念出来就是一个乱码字。

代码 4-8 按字符边界截断与语音模块的帧封装（源码/ESP32 嵌入式开发/智能语音_7 号 /src/tts.cpp, gb2312_safe_len() 与 play_now() 节选）

```

static unsigned int gb2312_safe_len(const String &s, unsigned int max_bytes)
{
    unsigned int i = 0;
    unsigned int total = (unsigned int)s.length();

    while (i < total && i < max_bytes)
    {
        unsigned char c = (unsigned char)s[i];
        unsigned int step = (c < 0x80) ? 1 : 2; // ASCII 单字节，汉字双字节
        if (i + step > max_bytes)
            break; // 再放就超了，到此为止
        i += step;
    }
    return i;
}

static void play_now(const char *text, uint8_t level)
{
    // 1) UTF-8 → GB2312。语音模块只认 GB2312，少这一步模块收到的
    // 是乱码，现象是"有声音但念的是杂音"，比完全不响更难查。
    String gb = GB.get(String(text));

    // 2) 超长截断，切在字符边界上

```

```

unsigned int len = gb2312_safe_len(gb, TTS_MAX_BYTES);

// 3) 组帧: 0xFD | 长度高字节 | 长度低字节 | 0x01 | 0x01 | GB2312 文本
// 长度字段 = 文本字节数 + 2, 填的是【字节数】不是字符数:
// 一个汉字占 2 字节, "欢迎" 是 4 字节, 长度字段写 4 + 2 = 6。
static uint8_t buf[TTS_MAX_BYTES + 8];
buf[0] = 0xFD;
buf[1] = (uint8_t)((len + 2) >> 8);
buf[2] = (uint8_t)((len + 2) & 0xFF);
buf[3] = 0x01; // cmd
buf[4] = 0x01; // para
if (len > 0)
    memcpy(buf + 5, gb.c_str(), len);
Serial2.write(buf, len + 5);

// ..... (略去播报时长估算与串口日志) .....
}

```

帧里的长度字段填的是字节数而不是字符数:「欢迎」是 2 个汉字共 4 字节, 长度字段写 4+2=6。这一段是调试中最容易出错的地方——波特率或校验位写错、长度字段填成字符数, 现象都是「完全没声音」或「念的是杂音」并且不报任何错, 所以相关参数与注释在 my_config.h 里都做了明确标注。

第二个关键点是播报队列。八块板合起来每秒会产生一到三条报文, 而语音念一句话要两三秒, 报文产生的速度是播报能力的数倍; 如果来一条念一条, 喇叭就会卡在同一句上反复念。因此队列带优先级: 高等级抢断低等级, 同等级排队, 队列满时丢弃等级最低的一条, 内容完全相同且正在播报的句子不重复入队。

代码 4-9 播报队列的查重与「队列满丢最低等级」策略 (源码/ESP32 嵌入式开发/智能语音_7 号/src/tts.cpp, enqueue() 节选)

```

static bool enqueue(const char *text, uint8_t level)
{
    for (int i = 0; i < q_count; i++)
    {
        if (strcmp(queue[i].text, text) == 0)
        {
            Serial.printf("[tts] 队列里已有同样的内容, 不重复入队: %s\n", text);
            return true;
        }
    }
    if (play_level != 255 && strcmp(playing_text, text) == 0)
    {
        Serial.printf("[tts] 正在播报同样的内容, 不重复入队: %s\n", text);
        return true;
    }
}

```

```

if (q_count < TTS_QUEUE_LEN)
{
    strncpy(queue[q_count].text, text, TTS_TEXT_MAX - 1);
    queue[q_count].text[TTS_TEXT_MAX - 1] = '\0';
    queue[q_count].level = level;
    q_count++;
    return true;
}

// 队列满：找等级最低的一条
int worst = 0;
for (int i = 1; i < TTS_QUEUE_LEN; i++)
    if (queue[i].level < queue[worst].level)
        worst = i;

if (level > queue[worst].level)
{
    uint8_t old = queue[worst].level;
    strncpy(queue[worst].text, text, TTS_TEXT_MAX - 1);
    queue[worst].text[TTS_TEXT_MAX - 1] = '\0';
    queue[worst].level = level;
    // ..... (略去串口日志) .....
    return true;
}
return false;
}

```

等级取值上，平台命令用 4，高于任何本地告警，保证人点的那一下一定能盖过联动播报；紧急（火焰）3、警告（遮挡）2、提示 1、恢复 0。tts_loop() 每次从队列里挑等级最高的一条，若它高于正在播报的等级就直接下发，语音模块收到新帧会自动切换，不需要先发停止命令。另外，arduino-mqtt 明确规定在消息回调里调用 publish / subscribe 会死锁，所以回调链上只做「判断 + 入队」，真正的串口下发统一放在 tts_loop() 里做。

在平台下发这条标准路径之外，7 号板还实现了三层本地播报结构，这也是本模块区别于其他执行器的地方，结构如图 4-3 所示。第一层是事件即时播报，状态一变立刻播，覆盖火焰、遮挡、设备上下线、各执行器的联动动作以及平台命令的舵机回执；第二层是阈值告警，温度、湿度、光照越界时播报，并带冷却时间防止刷屏；第三层是周期数据汇总，每 30 秒把各模块最新值合成一句话念出来。

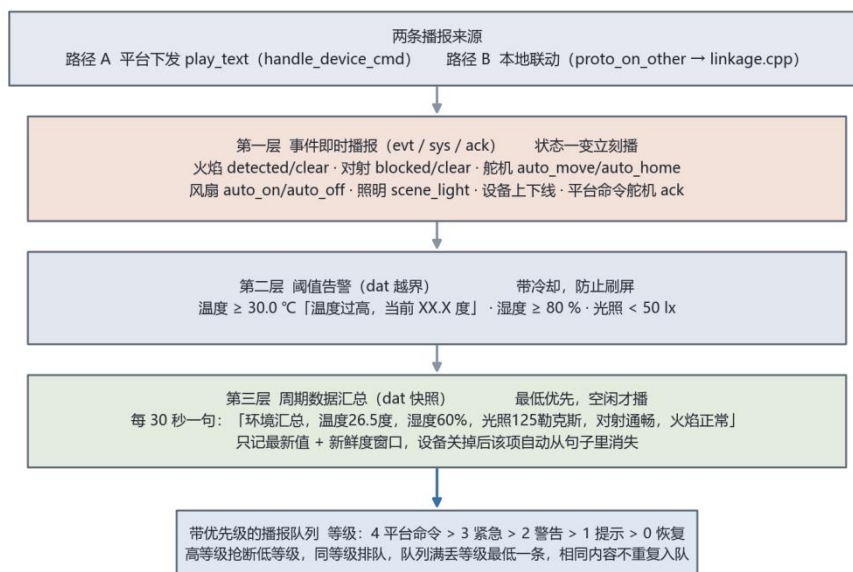


图 4-3 7 号板的三层播报结构

第三层是本次设计的重点，原因是前两层都只在「出了事」的时候说话，正常范围内一句都不播，于是听不到系统整体的运行状况。但逐条播报在物理上做不到：数据产生的速度是播报能力的数倍，排队只会越播越滞后。所以第三层不做队列，只做快照——每个数据源只记最新值，新数据来了覆盖旧的，到点合成一句念出来，念的永远是当前值。

代码 4-10 周期汇总的「数据快照 + 新鲜度判断」（源码/ESP32 嵌入式开发/智能语音_7号/src/linkage.cpp, build_summary() 节选）

```

static void build_summary(char *out, size_t n)
{
    char num[16];
    int pos = 0;

    // 先判断“哪几项还新鲜”。设备关掉之后不再上报，对应的时间戳就不再
    // 刷新，超过窗口它的那一项会自动从句子里消失，不会一直念旧数据。
    bool has_temp = snap_valid(snap.temp_ms, SNAP_TIMEOUT_SHT30_MS);
    bool has_humi = snap_valid(snap.humi_ms, SNAP_TIMEOUT_SHT30_MS);

    pos = appendf(out, n, pos, "环境汇总");

    if (has_temp && has_humi)
    {
        fmt_num(num, sizeof(num), snap.temp);
        pos = appendf(out, n, pos, ", 温度%s 度，湿度%d%%",
                      num, (int)(snap.humi + 0.5f));
    }
    else if (has_temp)

```

```

{
    fmt_num(num, sizeof(num), snap.temp);
    pos = appendf(out, n, pos, ", 温度%s 度", num);
}
else if (has_humi)
{
    pos = appendf(out, n, pos, ", 湿度%d%", (int)(snap.humi + 0.5f));
}

if (snap_valid(snap.lux_ms, SNAP_TIMEOUT_LIGHT_MS))
    pos = appendf(out, n, pos, ", 光照%d 勒克斯", (int)(snap.lux + 0.5f));

// ..... (对射与火焰两项的拼接与上面同理, 此处略) .....

out[n - 1] = '\0';
}

```

快照必须配一个新鲜度窗口：设备断电之后不再上报，如果只记一个「收到过没有」的标记，汇总播报会一直念一块已经断电的板子最后报上来的数据，听起来像它还在工作。所以每个数据源记的是时间戳，超过窗口（按各模块自身上报周期的两倍以上设置）就把这一项从句子中去掉，没有数据的模块整段省略，不念「未知」。

此外还有两处细节体现了「从现象倒推实现」的过程。一是火焰按轮次播报而不是按报文条数：6 号板在火焰持续期间每 5 秒重发一次 detected 以防平台漏报，那些重发不是新事件，如果拿一个较长的冷却时间压它们，代价是演示时点火常常不响；改成记住「本轮已播过告警」，同一轮里只按提醒间隔再念一次，收到 clear 本轮结束并重新武装，灭火后重新点火就能立刻播报。

代码 4-11 火焰的「按轮次」播报判断（源码/ESP32 嵌入式开发/智能语音_7 号
/src/linkage.cpp, on_evt_flame()）

```

static void on_evt_flame(const char *ev)
{
    if (strcmp(ev, "detected") == 0)
    {
        uint32_t now = millis();

        // 本轮没播过（第一次点火，或中间收到过 clear），或者距上一条
        // detected 超过一个重发周期（火焰断过、只是 clear 丢了）→ 新一轮
        bool new_round = !flame_alarming ||
            (now - last_flame_rx_ms) > FLAME_RESEND_GAP_MS;
        last_flame_rx_ms = now;

        // 同一轮里没到提醒间隔的就是重发，不出声
        if (!new_round && (now - last_flame_say_ms) < FLAME_REMIND_MS)
        {

```

```

        Serial.println("[link] 同一轮火焰的重发上报，不重播");
        return;
    }
    // 新一轮走 3 秒的抖动地板，不走 30 秒的告警冷却，
    // 这样灭火后再点火立刻就能播。只有真的播出去了才置标记：
    // 被冷却或队列挡下的那次不记账，后面的重发还有机会补播。
    if (say_rule_cd(R_FLAME_ON, "警告，检测到火焰",
                    TTS_LEVEL_EMERGENCY, COOLDOWN_FLAME_MS))
    {
        flame_alarming = true;
        last_flame_say_ms = now;
    }
}
else if (strcmp(ev, "clear") == 0)
{
    // 本轮播过告警才播"已解除"，两者配对，抖动时不会来回念。
    if (flame_alarming)
    {
        flame_alarming = false;
        say_rule_cd(R_FLAME_OFF, "火焰警报已解除",
                    TTS_LEVEL_RECOVER, COOLDOWN_FLAME_MS);
    }
}
}
}

```

二是设备上下线用「状态表 + 合并播报」。online 主题带 Retain 标志，某块板掉线后服务器会一直保留它那条离线报文直到它重新上线，而语音板每次重新订阅都会把这些保留报文再收一遍；如果照着报文就播报，会出现「设备一接电，先播一句已离线、再播一句已上线」的倒序现象。因此板内维护一份各板在线状态表，只有「表里记着它在线、现在却收到 offline」才算真掉线。另外上下线不加冷却——它们每块板只发生一次，被冷却挡掉就永远补不回来；改成先登记、等 2 秒把一批合成一句：「3 号、5 号设备已上线」，八块板一起上电也只念一句。

4.5 传感器模块的整合与统一

除自己开发的两块板之外，本人在整合阶段重点处理了六块传感器板的两类共性问题和一类个性问题。共性问题的第一类是「不能把无效读数当成有效数据上报」。温湿度模块读 SHT30 时先查 I2C 有没有应答、字节数是不是正好 6、两个 CRC 是否通过，任一不过就跳过本周期；光照模块读到一半失败时同样跳过，而不会退回光敏电阻的换算分支；原因是悬空引脚读到的噪声会变成一条看起来正常的假数据发出去，这比少一条数据危险得多。

共性问题的第二类是「被去抖与限流挡下的事件不能丢」。以 6 号板火焰传感器为例，按协议要去抖 50 毫秒、同一触发事件 500 毫秒内不重复上报，但状态跳变一块板只发生一次：如果『火焰消失』正好落在限流窗口里被直接丢弃，而状态已经改成无火，就再也不会重发，5 号板的声光报警会一直停不下来。因此代码把被限流挡下的跳变记下来，窗口一过立刻补发。

代码 4-12 火焰的「相对基线判定」与「被限流事件补发」(源码/ESP32 嵌入式开发/火焰_6号/src/flame.cpp, `calibrate()` 与 `scan()` 节选)

```
// — flame.cpp · calibrate(): 开机采样 2 秒取平均, 得到无火基线 —
long sum = 0;
int n = 0;
unsigned long t0 = millis();
while (millis() - t0 < FLAME_CALIBRATE_MS)
{
    sum += analogRead(PIN_FLAME_A0);
    n++;
    delay(20);
}

base = (int)(sum / n);

// — flame.cpp · scan(): 去抖、限流与被限流事件的记录 —
// ---- 去抖: 原始判定变化后重新计时 ----
if (raw != raw_last)
{
    raw_last = raw;
    raw_since_ms = now;
    return;
}
if (now - raw_since_ms < FLAME_DEBOUNCE_MS)
    return;

if (raw != flame_state)
{
    flame_state = raw;

    // 限流: 同一触发事件 500ms 内不重复上报
    if (now - last_evt_ms >= FLAME_LIMIT_MS)
    {
        last_evt_ms = now;
        send_state_evt(flame_state, ao, diff);
    }
    else
    {
        // 落在限流窗口里: 不丢, 记下来等窗口过去补发。
        // 期间状态若再变, 会覆盖这里的记录, 所以补发的永远是最新状态。
        pending_evt = true;
        pending_state = flame_state;
    }
}
```

```
}
```

个性问题来自 5 号板：它同时挂载 RGB 与蜂鸣器，并且参与两条板端联动（光照照明与火焰报警），两条链路要抢同一组灯和同一个蜂鸣器。最初的写法是「收到火焰事件就把灯设成红色」，结果事件以不同顺序到达时最终状态会不一样，还会留下「火焰灭了灯却还红着」的残影。整合时改成状态重算：把火焰、光照、平台手动三个输入都记成状态，任一变化就整体重算一次场景，优先级为火焰报警 > 平台手动命令 > 光照照明；同时给火焰报警加了 15 秒超时兜底——6 号板在火焰持续期间每 5 秒重发一次 detected，本板连续 15 秒收不到任何火焰报文就自动解除声光报警，避免解除报文丢失导致一直报警。

4.6 配置集中与非阻塞的工程约定

八块板的 WiFi、服务器、主题、板号、引脚、阈值、时序参数与协议层开关，全部写在各自的 my_config.h 里，业务代码一行都不碰这些常量。现场换热点、调整阈值或改接引脚，只需改这一个文件。（WiFi 账号密码按现场实际使用的热点填写，下面这段摘录以课程实验箱所在教室的热点为例。）以 1 号板为例，它的配置中与硬件和联动直接相关的部分如下。

代码 4-13 1 号板的集中配置（源码/ESP32 嵌入式开发/舵机_1 号/src/my_config.h 节选）

```
#define WIFI_SSID      "IOT-SZCS01"          // WiFi 账号
#define WIFI_PASSWORD "iot-szcs01"          // WiFi 密码
#define MQTT_SERVER   "yunyismart.tech"      // MQTT 服务器地址
#define MQTT_PORT      1883                  // MQTT 服务器端口号

// 前缀 = 课程空间前缀 202609_PP2 + 组号 1
#define GROUP_ID 1
#define TOPIC_CMD "202609_PP2/1/cmd"        // 下行：各平台 → 各板（本板只订阅这个）
#define TOPIC_REPORT "202609_PP2/1/report"   // 上行：各板 → 各平台
#define TOPIC_ONLINE "202609_PP2/1/online"   // 上行：上线下线，Retain

#define BOARD_ID 1
#define BOARD_ID_STR "1"
#define DEV_SERVO "servo"

#define SERVO_PIN 14    // 舵机信号线（课堂例程的接法）
#define PIN_LED1 15     // 板载 LED：转动期间点亮，到位熄灭（低电平点亮）
#define SERVO_ANGLE_MIN 0
#define SERVO_ANGLE_MAX 180
#define SERVO_HOME_DEG 90 // 上电自动回到的中位角度

// 转动被拆成「每 SERVO_STEP_MS 毫秒走 SERVO_STEP_DEG 度」的轨迹，
// 全程耗时 = 角度差 / 步长 × 步间隔，0° → 180° 约 1.8 秒。
#define SERVO_STEP_DEG 1 // 每步走多少度
```

```

#define SERVO_STEP_MS 10 // 每步间隔多少毫秒

// 3 号板检测到光束被遮挡时，本板自动把舵机转过去；光束恢复后转回原位。
#define LINK_SERVO_BLOCKED_DEG 0 // 遮挡时转到 (0°)
#define LINK_SERVO_CLEAR_DEG 90 // 恢复时回到 (中位 90°)

// 协议层开关 (定义见 mqtt_proto.h)
#define ACK_AFTER_DONE 1 // 转动到位之后才回执 (协议表 19)
#define SUB_EXTRA_REPORT 1 // 联动旁听: 接收 3 号板的遮挡事件
#define SUB_EXTRA_ONLINE 0 // 本板不订阅 online
#define MQTT_BUF_SIZE 1024 // MQTT 接收缓冲区

```

这里有一处实际踩到的坑值得记录：配置宏不能命名为 SSID / PASSWORD。ESP32 的 WiFi.h 里有一个方法就叫 SSID()，一旦写成 #define SSID "...", 那个方法的声明会被宏替换掉，编译直接报 expected unqualified-id before string constant。课堂例程没有出这个问题，是因为它把 #include <WiFi.h> 写在 #include "my_config.h" 之前；而本工程的 mqtt_proto.h 是先包含 my_config.h 的，所以必须改用 WIFI_SSID / WIFI_PASSWORD，不能依赖包含顺序。

最后一组贯穿全部八块板的约定是「不阻塞主循环」。主循环里跑着 MQTT 心跳与收发，任何一处卡住都会导致收不到命令、回执延迟，卡久了还会被服务器判超时踢下线，因此舵机的平滑转动、蜂鸣器的定时鸣叫、火焰的持续重发、语音的播报队列、对射的心跳全部用时间戳比较实现，没有一处 delay 阻塞。唯一例外是火焰板开机时的 2 秒基线校准，那发生在上电初始化阶段，此时还没有需要及时响应的报文，而且 2 秒远小于 MQTT 的 60 秒 keepalive，不会引起掉线。

第五章 基于 App Inventor 的安卓程序设计

安卓端是本组系统的「监控台 + 遥控器」，由组内成员使用课程指定的 MIT App Inventor 开发。它与 Web 端使用同一套主题与报文格式，只与 MQTT 服务器通信，与 ESP32 完全解耦；提交物为可编译的 group01_final.aia、安装包 group01_final.apk 以及一份程序功能与设计说明。

5.1 界面构成

界面按「看」与「控」分开的原则做成三个页面，底部（顶部）一行三个按钮切换：仪表盘用于看数据与状态，控制页用于操作五类执行器，系统页显示收到的全部原始报文用于排查问题。这样划分的原因是现场演示时这两件事几乎不会同时做——看数据时界面要干净、不误触按钮；操作时又要一屏就能触到所有控件。第三个页面平时用不到，属于调试工具，所以放在最后。

仪表盘上的实测界面如图 5-1 所示。环境读数做成五个卡片式读数（温度、湿度、光照、火焰强度、对射状态），各占一半宽、居中放大显示；系统状态区显示已识别板数、最近一条回执与最近一次联动；近期事件列表带时间戳，最新的在最上面；最下面是六个「模拟传感器」按钮。图中读数显示温度为 27.3 °C、湿度 73.1 %RH、光照 204.2 lx，已识别板为 6/8，最近一次联动是 rgb 的 scene_fire。

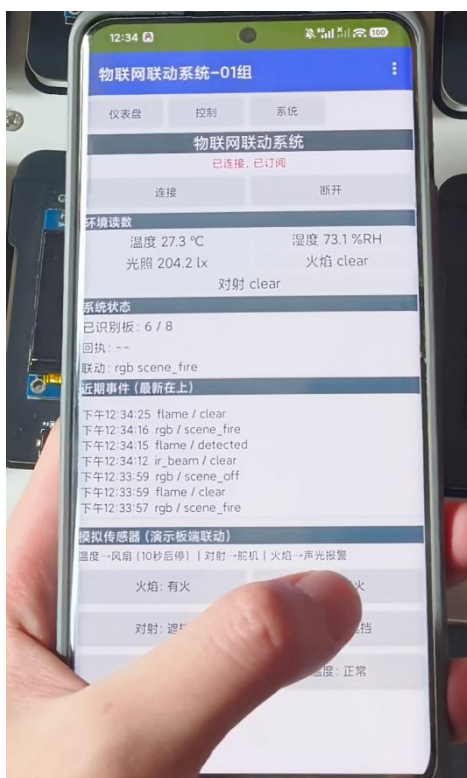


图 5-1 安卓端「仪表盘」页面的实测界面

这里有两处与协议特性直接相关的设计。一是「已识别板数」而不是「在线板数」：MQTT 的 Retain 机制是每个主题只保留最后一条消息，而八块板发的是同一个 online 主题，所以 App 一连上只能收到 1 条保留的上线报文，靠它数板子数量永远是 1。因此改成「本次连接期间收到过任何报文的板都算已识别」，收到报文就按 src 记录板号并去重累加，这个说法是准确的。二是连接默认是断开、需要手动点击连接：最初做的是上电自

动连接，但开机瞬间往往还在接线、摆板子，自动连上去以后一会儿掉线一会儿重连，反而不如手动连接时状态清楚。

最下面一排「模拟传感器」按钮是本端最有演示价值的设计：六个按钮分成三对，每对是「触发 / 解除」（火焰：有火 / 无火，对射：遮挡 / 无遮挡，温度：偏高 / 正常）。点一下，安卓端会往 report 主题发一条假的上报报文，报文里的 src 填被模拟的那块板的号（火焰填 "6"、对射填 "3"、温度填 "4"）。执行器板只按 src 过滤「是不是自己发的」，不区分报文来自真板子还是模拟器，因此照常触发板端联动。这正是协议 § 9.3 建议的做法（原文是用 MQTTX 手动发报文），本组把它直接做进了 App 里——演示时不用打火机、不用手挡光路，点一下按钮就有，而且可以重复、可以远程。

5.2 逻辑块设计

安卓端一共使用了 35 个事件块与 34 个方法块，全部组件的程序块如表 5-1 所示，按功能整理后的结构如图 5-2 所示。

表 5-1 group01_final.aia 的程序块清单

分类	程序块	作用
初始化	Screen1.Initialize	设置界面初始文本与 24 个全局变量初值
连接	btn_connect.Click → Connect	以固定客户端编号连接 MQTT 服务器
连接	btn_disconnect.Click → Disconnect	主动断开连接
连接	ConnectionStateChanged	连接状态变化时更新界面提示
收发	MessageReceived	收到报文后取主题与正文，转字典并分流
收发	PublishEx（共 23 处）	下发命令与联动模拟报文，QoS 1
参数输入	sld_servo、sld_r、sld_g、sld_b 的 PositionChanged	滑块数值实时显示
执行器控制	btn_servo0/45/90/135/180、btn_servo_go	舵机角度快捷键与按滑块角度转动
执行器控制	btn_fan_on、btn_fan_off	风扇开 / 关
执行器控制	btn_red、btn_green、btn_blue、btn_warm、btn_light_off、btn_rgb_go	RGB 预设色、灯灭与按滑块设色
执行器控制	btn_beep、btn_buzz_off	蜂鸣器按时长鸣叫 / 停止
执行器控制	btn_tts	播报文本框中的内容
联动演示	btn_fire_on/off、btn_ir_blocked/clear、btn_temp_high/low	向 report 主题发布模拟上报报文

分类	程序块	作用
页面切换	btn_nav1、btn_nav2、 btn_nav3	切换仪表盘 / 控制 / 系统三个页面



图 5-2 安卓端逻辑块的结构 (按 group01_final.aia 的积木清单整理)

报文解析采用「网页」组件的 `JsonTextDecodeWithDictionaries` 把报文文本转成字典, 再按 `type` 分成 `dat`、`evt`、`ack`、`sys` 四条分支: `dat` 更新对应的读数卡片, `evt` 写进近期事件并更新器件状态, `ack` 显示「器件 动作 → ok/fail」, `sys` 更新已识别板数并写上下线事件。发送侧则不做字典序列化, 而是把报文拼成文本后直接发出——这样做是为了精确控制字段顺序、`seq` 的数值以及「数值不加引号」这条约定; 拼装多参数报文 (例如 RGB 的三个通道) 时字段之间的逗号必须写全, 少一个整条报文就不是合法 JSON, 板子会直接丢弃, 这是开发中实际踩过的坑, 最终版所有报文都做过「还原成 JSON 再解析一遍」的自动校验。

发布块的选择也是协议要求带来的: App Inventor 的 MQTT 扩展有两个发布块, `Publish` 只有主题与消息两个参数、`QoS` 固定为 0, `PublishEx` 有四个参数、可以指定保留标志与 `QoS`。协议表 18 要求 `cmd`、`evt`、`ack` 使用 `QoS 1`, 而课程提供的参考工程用的是 `Publish`, 照抄的话所有命令都会走 `QoS 0`——报文看着发出去了, 但服务器不做重试, 板子会偶尔收不到命令。因此本端所有下发一律使用 `PublishEx`, 并显式传 `QoS = 1`、保留 = 假。

第六章 Web 端的程序设计

Web 端是本组系统的另一个控制台，同样由组内成员开发，提交物为 index.html、style.css 与 script.js 三个文件，使用 MQTT.js 库通过 WebSocket 连接同一台 MQTT 服务器（端口 9001）。它不依赖任何构建工具，双击 index.html 即可在浏览器中运行，调试成本低，是现场演示时的主要展示界面。

6.1 页面结构

页面自下而上分四个区域：顶部的连接状态栏与最近消息、设备在线状态网格、按板号排列的八张设备卡片、最下方的联动调试面板与消息日志。实测界面如图 6-1 所示。

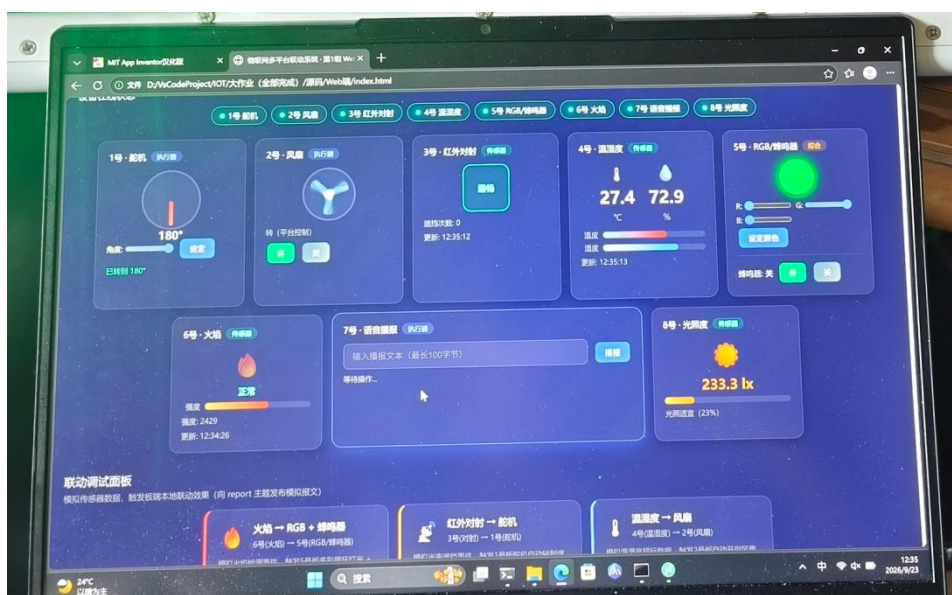


图 6-1 Web 控制台界面（浏览器中运行，含八块设备卡片与联动调试面板）

八张设备卡片与 8 块板一一对应，这是本页面排版上最重要的一条原则：同一块板、同一个器件的东西全部放在自己那一组里。卡片分成「执行器」「传感器」「综合」三类，标题上带角标；执行器卡片带控制控件，传感器卡片只显示读数与进度条，并按数据类型给出辅助图形——舵机画了一个随角度旋转的机械臂，风扇用 CSS 动画表现转动，RGB 卡片用一块色块预览当前颜色，光照卡片按照亮度给出「黑暗 / 较暗 / 光照适宜 / 强光」四档提示。

6.2 MQTT 连接与报文订阅

连接与订阅在 connectMQTT() 中完成。客户端编号带随机后缀，避免同一台机器开多个页面时互相顶下线；连接成功后订阅 report 与 online 两个主题，与设备端的订阅范围正好错开——设备端只订阅 cmd，两端因此不会互相收到自己发出的报文。

代码 6-1 Web 端的 MQTT 连接与订阅（源码/web 端开发/script.js，connectMQTT() 节选）

```
const HOST = "yunyismart.tech";
const PORT = 9001; // WebSocket 端口
const TOPIC_CMD = "202609_PP2/1/cmd";
```

```

const TOPIC_REPORT = "202609_PP2/1/report";
const TOPIC_ONLINE = "202609_PP2/1/online";

function connectMQTT() {
  const url = "ws://" + HOST + ":" + PORT;
  mqttClient = mqtt.connect(url, {
    clientId: "web_" + Math.random().toString(16).substr(2, 8),
    clean: true,
    reconnectPeriod: 2000
  });

  mqttClient.on("connect", function () {
    // 订阅 report（传感器数据、事件、回执）和 online（上下线）
    mqttClient.subscribe(TOPIC_REPORT, { qos: 1 });
    mqttClient.subscribe(TOPIC_ONLINE, { qos: 1 });
  });

  mqttClient.on("message", function (topic, payload) {
    const msg = payload.toString();
    try {
      const data = JSON.parse(msg);
      handleReport(data, topic);
    } catch (e) {
      // 非 JSON 忽略
    }
  });
}

```

6.3 命令下发与界面更新

所有命令都由一个 `sendCmd()` 统一封装信封，各器件的按钮只负责取参数并调用它。这样「信封格式」这件事在代码里只有一处实现，协议若要升版只改这里。

代码 6-2 命令信封的统一封装（源码/web 端开发/script.js, `sendCmd()`）

```

function sendCmd(dst, device, action, param) {
  if (!mqttClient || !mqttClient.connected) {
    alert("未连接 MQTT 服务器");
    return;
  }

  const msg = {
    ver: "1.0",
    type: "cmd",
    seq: txSeq++,

```

```

    src: "web",
    dst: dst,
    body: { device: device, action: action }
  };
  if (param && Object.keys(param).length > 0) {
    msg.body.param = param;
  }
  if (txSeq > 65535) txSeq = 1;

  const payload = JSON.stringify(msg);
  mqttClient.publish(TOPIC_CMD, payload, { qos: 1 });
  addLog("send", "[web -> " + dst + "号] " + payload);
}

```

收到报文后的处理按 type 分流：dat 更新读数与进度条，evt 更新器件状态，ack 把回执结果显示在对应卡片上（例如舵机显示「已转到 90°」、参数非法时显示「设定失败」），sys 更新在线状态网格。其中有一类细节值得单独提出：5 号板因联动而改变灯光时，发的是扩展事件 scene_light / scene_fire / scene_off，Web 端收到后会 RGB 滑块的数值同步过去，并在日志里写明「5 号板联动：光照照明 → RGB(255,200,120)」，让使用者看得见这是联动触发而不是自己点的。

6.4 联动调试面板

页面最下方是联动调试面板，与安卓端的「模拟传感器」对应，但按联动链路组织成三张卡片：火焰 → RGB + 蜂鸣器（6 号板 → 5 号板）、红外对射 → 舵机（3 号板 → 1 号板）、温湿度 → 风扇（4 号板 → 2 号板），每张卡片两个按钮。点击后 Web 端会往 report 主题发布一条 src 为被模拟板号的报文，同时把界面上的对应状态也更新一遍，因此不需要真实传感器就能演示整条联动链路。

需要特别注意的是，模拟报文必须发到 report 主题而不是 cmd：执行器板是订阅 report 来旁听传感器上报的，发到 cmd 不会触发联动；同时报文里的 src 必须填被模拟板的号，否则会被板子当成「别人发的事件」而在目标板上找不到对应的处理逻辑。这两条都来自板端联动的实现方式，也是协议 §9.3 明确写出的调试要求。

第七章 系统联调与实验结果

7.1 联调环境与步骤

联调在实验箱上完成：8 块 ESP32 模块插在底板上，由实验箱统一供电，全部连接到同一 Wi-Fi 与 MQTT 服务器 `yunyismart.tech`；Web 端运行在笔记本电脑的浏览器中，安卓端运行在手机上，两端都通过公网连到同一台服务器，与设备不在同一个局域网。联调按「先单板、后整组」的顺序进行：先用 MQTTX 对单块板逐条验证命令与上报，确认无误后再把八块板一起上电；最后再接入 Web 端与安卓端做多平台验证。

7.2 实验一：八块板同时在线

八块板依次上电后，各自连接 WiFi 与 MQTT 服务器，并向 `online` 主题发布上线报文。实测结果如图 7-1 所示：实验箱上八块板全部上电（面板上的电源指示与各板指示灯均亮），Web 端的在线状态网格与安卓端的已识别板数同时反映出多块板在线。



图 7-1 实验箱上 8 块 ESP32 模块同时在线

这一步验证了三件事：八块板使用的是各自唯一的客户端编号（格式为 `esp32- 加板号加 MAC 地址`），没有出现互相顶下线；每块板都正确订阅了本组的 `cmd` 主题；`online` 主题的 `Retain` 设置生效，平台端订阅后能立刻读到设备状态，不必等待下一次上线报文。

7.3 实验二：多平台控制与状态回填

本实验验证「平台下发命令—设备执行—回执回到平台」这条闭环。测试项覆盖五类执行器，结果如表 7-1 所示。其中舵机一项特意验证了延迟回执：在 Web 端把角度设为 180° 并点击设定，界面先显示「正在转动到 $180^\circ \dots$ 」，约 1.8 秒后收到回执并改显示「已转到 180° 」，与代码 4-4、代码 4-6 的设计一致；把角度设为 200° （越界）时，回执为 `fail`，界面显示「设定失败（参数非法）」，舵机保持原位不动。

表 7-1 多平台控制与回执实测结果

测试项	下发内容	预期现象	实测结果
舵机角度	dst=1, set_angle, angle=180	转动约 1.8 s 后回执 ok, param 回发 180	符合
舵机越界	dst=1, set_angle, angle=200	回执 fail, 舵机不动	符合
舵机非整数	dst=1, set_angle, angle=90.5	回执 fail, 舵机不动	符合
风扇开关	dst=2, set_power, on=true / false	继电器吸合 / 断开并回执 ok	符合
RGB 设色	dst=5, set_color, r=g=b=0	四颗 WS2812 熄灭并回执颜色	符合
蜂鸣器	dst=5, set_power / beep(duration=500)	持续鸣叫 / 按时长鸣叫后自动停	符合
语音播报	dst=7, play_text, text="欢迎使用物联网系统"	模块用中文播报, 回执 ok	符合
广播命令	dst=0, rgb set_color	各板按各自实现的设备响应, 未 实现的忽略	符合

安卓端与 Web 端在这一环节的表现一致：两端都订阅了 report 主题，因此同一条回执会同时到达两个平台并各自更新界面。实测中发现一处需要说明的现象：安卓端连接得比 Web 端晚时，其「已识别板数」会从 0 开始重新累计，这是第二节提到的 Retain 机制导致的局限（同一主题只保留最后一条消息），已在程序功能与设计说明中作为已知问题记录。

7.4 实验三：板端联动实测

板端联动是本组相对协议第七章方案的个性化实现，四条规则全部做了实测，结果如表 7-2 所示。测试方式分两类：温湿度、光照、对射三条用真实传感器造条件；火焰一条用打火机靠近 6 号板的红外接收管，如图 7-2 所示。



图 7-2 火焰联动实测：打火机靠近 6 号板，Web 端同步显示火焰状态

表 7-2 四条板端联动的实测结果

联动规则	造条件方式	预期动作	实测结果
4 号温湿度 → 2 号风扇	对传感器哈气提高湿度至 80 % 以上	风扇自动开启；湿度回落到 75 % 以下并持续 10 秒后关闭	符合，10 秒延时关闭正常
3 号对射 → 1 号舵机	用手遮挡光路	舵机立即转到 0° ；移开手后回到 90°	符合，响应在 1 秒以内
8 号光照 → 5 号 RGB	用手遮住 BH1750 使读数降到 50 lx 以下	点亮暖白光（255,200,120）；照度升到 120 lx 以上后熄灭	符合
6 号火焰 → 5 号 RGB + 蜂鸣器	打火机靠近火焰传感器	RGB 多彩循环 + 蜂鸣器间歇报警；移开后解除	符合，解除后按光照重新决定

图 7-3 是火焰报警的现场照片：6 号板检测到火焰后，5 号板的 RGB 进入报警场景、蜂鸣器间歇鸣叫，同时安卓端的「近期事件」列表按时间顺序记录下了这一串事件（rgb/scene_fire、flame/detected、随后 flame/clear 与 rgb/scene_off），而「联动」一栏显示最近一次联动为 rgb scene_fire，说明平台侧确实能区分「联动触发」与「人工操作」。



图 7-3 火焰报警的声光联动与安卓端事件记录

图 7-4 是光照联动的现场照片：遮住 8 号板的光照传感器使读数低于阈值后，5 号板自动点亮暖白光照明，Web 端的光照卡片同步显示当前照度与「黑暗 / 较暗」分档，整个过程中没有任何平台命令参与。



图 7-4 光照不足时 5 号板自动照明（板端联动，未经平台）

这几条链路验证了把联动放在设备端的价值：遮挡到舵机动作的延迟在 1 秒以内，明显快于「上报—平台判断—下发」的往返；并且这些行为不依赖平台是否在线。同时，执行器完成联动后发出的扩展事件又让平台与 7 号板能够感知到动作的发生，兼顾了「快」与「可见」。

7.5 实验四：语音三层播报

7 号板的三层播报做了单独验证。第一层：点火后立即播报「警告，检测到火焰」，移开打火机后播报「火焰警报已解除」；遮挡对射时播报「注意，检测到遮挡」，并伴随 1 号板的联动转动播报「舵机已转到遮挡位」。第二层：把传感器加热到 30 °C 以上时播报「温度过高，当前 XX.X 度」。第三层：在系统稳定运行的空闲时段，每 30 秒播报一句汇总，内容形如「环境汇总，温度 26.5 度，湿度 60%，光照 125 勒克斯，对射通畅，火焰正常」。

验证过程中重点确认了两条工程约定确实生效：一是播报不排队堆积——在连续触发多条告警时，汇总播报会自动让路，一空闲立刻补上，不会出现「念的是几分钟前的数据」；二是设备断电后其数据会从句子里消失——关闭 4 号板约 15 秒后再听汇总，句子中不再出现温度与湿度，而是从光照开始念，整句依然通顺。

7.6 结果分析

从四次实验的结果看，系统达到了设计目标：8 块板可以同时在线并被两个平台观察；五类执行器都能被远程控制且回执正确；四条板端联动在 1 秒以内响应，四条规则对应的物理动作全部符合预期；语音播报的三层结构都能在合适的时机说话，并且不会因为数据过快而滞后。

仍然存在的问题主要有三个。第一，online 主题的 Retain 机制在各板共用一个主题时只能保留最后一条消息，导致后连接的平台无法完整还原全部设备的在线状态，目前的应对是改用「本次连接期间见过哪些板」来统计，要彻底解决需要协议层面调整（例如每块板发到自己的 online 子主题，或平台周期性拉取）。第二，MQTT 的 QoS 1 是「至少一次」语义，同一事件可能被重复投递，本组在 7 号板里用「内容查重 + 状态表」解决了由此引起的重复播报，但其他板上仍是依靠传感器自身的限流窗口来抑制重复。第三，火焰传感器的基线校准依赖上电瞬间的环境，若校准期间有火源，基线会被抬高，需要断电重启重新校准，这是相对阈值判定法固有的代价。

第八章 总结

8.1 主要工作与收获

本次课程设计中，本人独立完成了《MQTT 通信协议规范_第1组》V2.0 的编写与修订，开发了 1 号板舵机与 7 号板智能语音播报两块板的固件，并负责其余六块板固件的整合与统一管理。系统最终实现了「8 块 ESP32 + Web 端 + 安卓端」的多平台联动，四条板端联动规则与三层语音播报都能在现场稳定演示。

收获最大的地方是协议设计与动手实现之间的关系。协议看起来只是几个字段和几条主题，但每一个决定都会代码里变成具体的约束：cmd 不保留是为了避免重连后误动作，所以设备端必须自己做「上电补报当前状态」；报告与回执必须用 QoS 1，所以课程例程使用的库做不到，必须换成支持指定 QoS 的库，而换库又牵出了缓冲区必须放大这个隐蔽的坑；online 用 Retain 让平台能立即读到状态，却因为各板共用一个主题而只能保留最后一条，这个矛盾最后是由平台侧改用「已识别板数」来绕开的。换句话说，协议设计不是写文档，而是在为后面所有平台的实现方式做决策。

第二个收获来自「执行器与传感器对错误的容忍度完全不同」这件事。传感器偶尔少报一条数据，影响只是曲线缺一个点；而执行器误动作可能打坏机构。因此本组对所有执行器命令都执行同一条原则——参数缺失、类型不对、越界一律回 fail 并保持原状态，绝不猜测用户的意图。这条原则在代码里表现为：参数校验与动作执行必须在同一步完成，校验不过就不进入执行分支。

8.2 遇到的问题与解决办法

开发过程中遇到的问题与解决办法整理如表 8-1 所示。这些问题大多不是「写错了」而是「没意识到」，记录下来对后续类似项目有参考价值。

表 8-1 开发中遇到的主要问题与解决办法

问题现象	原因	解决办法
带 param 的报文到达板子后毫无反应，串口无任何输出	arduino-mqtt 收发缓冲区默认仅 128 字节，长报文被静默截断	构造客户端时显式放大缓冲区： MQTTClient client(1024, 1024)
按协议开头的宏名 SSID 定义 WiFi 账号导致编译失败	ESP32 的 WiFi.h 中有同名方法 SSID()，宏替换掉了方法声明	宏名改为 WIFI_SSID / WIFI_PASSWORD，不依赖包含顺序
火焰熄灭后 5 号板的声光报警停不下来	clear 状态跳变落在 500 ms 限流窗口里被丢弃，此后再无机会重发	被限流挡下的跳变记下来，窗口过后立即补发
设备一接电，语音先播「已离线」再播「已上线」	online 主题带 Retain，服务器保留的是它上次掉线的旧报文	板内维护在线状态表，只有「表里在线却收到 offline」才算真掉线
演示时点火经常不播报	用 30 秒冷却去压火焰的 5 秒重发，导致新一轮点火被当成重发	改为按「轮次」播报：收到 clear 本轮结束并重新武装

问题现象	原因	解决办法
汇总播报念的是几分钟前的数据	八块板每秒产生多条报文，逐条排队必然越来越滞后	第三层改为只记最新值的快照，不做队列
设备断电后汇总播报仍在念它最后的数据	快照只记「收到过没有」，设备不再上报时该标记不会失效	改为记时间戳，超过新鲜度窗口就把该项从句子里去掉
舵机平滑转动期间收不到新命令、回执延迟	用循环加 delay 一次转到位，阻塞主循环近 2 秒	拆成每 10 ms 走 1° 的非阻塞状态机
平台连点同一个角度按钮，语音重复播报	QoS 1 的重复投递与人为重复点击都会产生相同报文	队列入队前查重，并记录上一次已播报的角度

8.3 不足与改进方向

系统目前有三处不足，也对应三个改进方向。第一，在线状态的完整还原问题：由于八块板共用同一个 online 主题，Retain 只能保留最后一条消息，后连接的平台无法一次拿到全部设备的状态。改进的方向是把在线状态改到每块板各自的子主题（如 online/1、online/2），或由平台在连接后主动发一轮查询主题让各板重发状态。

第二，板端联动的规则目前写在固件里，改一条阈值就要重新烧录对应的板子。本次已经把参数集中到 my_config.h，但这只是第一步；更进一步的做法是把联动规则做成可以由平台下发的配置（例如新增一条 cmd，把阈值写进设备的非易失存储），这样现场调整就不需要连电脑。

第三，平台端目前只有即时数值显示，没有历史曲线。安卓端受 App Inventor 组件限制无法方便地画图，Web 端虽然有条件实现，但本次没有做数据持久化。若后续引入一个轻量的时序数据库或在服务端记录 report 报文，就可以在 Web 端补上趋势曲线，让「数据展示」这件事更完整。

总的来说，本次课程设计把课上分散在嵌入式、移动端、Web 端三部分的知识串成了一条完整的链路，也让人真正体会到：在多设备、多平台的系统中，清晰的协议与严格的工程约定，比任何一段聪明的代码都更重要。